서울시립대학교 · 전자전기컴퓨터설계실험II

LAB7 실험 전 보고서

STM32_3 — SPI로 MCU와 FPGA 연결 (과제8 MCU 송신 파형 · 과제9 MCU→MCU · 과제10 MCU→FPGA · 과제11 FPGA→MCU · 과제12 FPGA→FPGA)

이름	학번	조
형종희	2023440145	C조 (과제9·12 짝: D조)

작성일	실험일	FPGA 보드
2026-10-27	2026-11-02	Combo II-DLD S75 · xc7s75fgga484-1

MCU 보드	도구
NUCLEO-G474RE	Vivado 2026.1 (XSim) · Icarus Verilog 12.0 · STM32Cube (GCC)

📦 GitHub 제출 정보

항목	값
저장소	https://github.com/hyeongjonghee/electronics2-LABs (Public)
LAB7 설계 소스 커밋	`68e6369` — FPGA SPI 수신·송신 회로와 테스트벤치, Icarus/XSim 결과와 파형(VCD), 보드 빌드 보고서, STM32 송신·수신 코드와 PC 모의 실행 파일
LAB7 경로	`LAB7/circuits/LAB7-01~03/` , `LAB7/stm32/` , `LAB7/lab7_tools.ps1` , `LAB7/LAB7_실험전보고서.html` · `.pdf`
실험 전 보고서 태그	`lab7-prelab` — 이 보고서(HTML · PDF)가 들어간 커밋. 최종 제출 태그 `lab7-submit` 은 실험 후 보고서 커밋에 따로 건다.
STM32 기반 프로젝트	`Glaysia/ece2_474` 태그 `v1.1.0` = 커밋 `dd94f01` . 바꾼 파일은 `Core/Src/ece2_474_routine.c` 하나이며, 과제별 사본을 위 저장소 `LAB7/stm32/` 에 보관.

목차

1. 사전 결과 요약
2. 과제8 · MCU 송신 파형
3. 과제9 · MCU → MCU
4. 과제10 · MCU → FPGA 수신
5. 과제11 · FPGA → MCU 송신
6. 과제12 · FPGA → FPGA
7. 고찰
8. 참고문헌

0. 사전 결과 요약

LAB7은 11/2 수업(STM32_3 · SPI로 MCU와 FPGA 연결)의 실습이다. 수업 문서가 정한 **과제 5개(과제 8~12)**를 순서대로 모두 준비했다. MCU 쪽은 매뉴얼이 준 송신(마스터)·수신(슬레이브) 코드를 쓰고, **FPGA의 SPI 수신 회로와 송신 회로는 정답 코드 없이 직접 작성**했다. I^2C는 이번 실습 범위가 아니다.

SPI 규칙은 수업 통일안을 따른다. **모드 0**(SCK 대기 LOW, 수신 측은 SCK 상승 에지에서 읽고 송신 측은 하강 에지에서 다음 비트로 바꿈), **8비트, 높은 비트(MSB)부터**, CS가 LOW인 동안만 통신, MISO는 연결하지 않는다.

#	실험 (무엇을 하는가)	검증	결과	자원 / 빌드
1	**과제8** · MCU가 SPI로 1초마다 한 바이트를 보내고, 그 파형(SCK·MOSI·CS)을 오실로스코프로 본다	빌드 + PC 모의 실행	PASS SPI1 모드 0 · MSB · 781.25 kHz, CS LOW 안에서만 전송, TX=55·A5·00·FF	BUILD OK (송신판·네 값 순환판)
2	**과제9** · MCU 두 대(C조·D조)를 연결해, 한쪽이 보낸 값을 다른 쪽이 받아 PC 화면에 출력	빌드 + 송신 모의 출력 → 수신 모의 실행	PASS RX=55·A5·00·FF ×2, 송신이 끝나면 RX timeout	BUILD OK (수신판)
3	**과제10** · MCU가 보낸 SPI 바이트를 FPGA가 받아 FPGA UART로 PC에 "RX=55"처럼 출력	Icarus · XSim 자동 검사 (MCU 파형 모델)	PASS 16바이트 · 14줄, 덜 찬 프레임 버림, 4바이트 프레임, CS HIGH 잡음 무시, 1 MHz·100 kHz	LUT 160 · FF 141 · IOB 14 · **WNS +14.907 ns**
4	**과제11** · FPGA가 마스터가 되어 1초마다 DIP 스위치 값을 SPI로 MCU에 보낸다	Icarus · XSim 자동 검사 + 받은 바이트로 MCU 수신 모의 실행	PASS 모드 0 규칙 8가지, SCK 500 kHz, MCU 출력 RX=55·A5·00·FF·3C·81	LUT 207 · FF 202 · IOB 22 · **WNS +14.323 ns**
5	**과제12** · FPGA 두 대(C조·D조)를 연결해 과제11 회로가 보내고 과제10 회로가 받는다	Icarus · XSim 두 보드 시뮬레이션 (클록 ±100 ppm), 역할 교대	PASS 보드 A "TX=xx" = 보드 B "RX=xx" 5개 ×2방향	과제10·11 비트스트림 그대로

설계 결정 4가지

1. **받은 값을 사람이 읽는 글자로 출력**: FPGA는 받은 바이트를 `RX=55\r\n` 처럼 16진수 글자로 보낸다. MCU 수신 예제의 출력과 모양이 같아서 Serial Monitor 화면 두 개를 나란히 놓고 비교할 수 있다. 0x00 · 0xFF처럼 화면에 안 보이는 값도 그대로 확인된다.
2. **FIFO로 속도 차이 흡수**: SPI 한 바이트는 약 10 µs, UART 한 글자는 1.04 ms라 약 100배 느리다. 받은 바이트를 64칸 FIFO에 먼저 쌓고 UART가 빌 때마다 꺼내 보낸다.
3. **보낼 값 = DIP 스위치**: 과제11·12에서 0x55 · 0xA5 · 0x00 · 0xFF를 실험 당일 다시 빌드하지 않고 스위치로 바꾼다. MCU 송신도 네 값을 1초마다 차례로 보내는 판을 따로 준비했다.
4. **기말 프로젝트 재사용**: 수신기는 CS가 LOW인 채로 여러 바이트를 연속으로 받고(바이트 번호 출력), 송신기는 바이트 수를 매개변수(`NBYTES`)로 바꿀 수 있다. 프로젝트의 4바이트 레지스터 맵을 그대로 붙일 수 있게 했다.

검증 범위 FPGA 회로는 Icarus와 Vivado XSim 두 시뮬레이터에서 같은 결과를 냈다. MCU 코드는 실제 STM32 컴파일러로 빌드하고, HAL 함수를 흉내 낸 PC 모의 실행으로 동작 순서(CS LOW → 전송 → CS HIGH, 슬레이브 재설정)를 검사했다. 실제 핀의 전압·잡음·배선 지연은 실험 당일 오실로스코프와 영상으로 확인한다.

1. 실험 1 · STM32_3 과제8 — MCU 송신 파형 확인

MCU 한 대가 SPI로 1초마다 한 바이트를 보내고, 오실로스코프로 SCK · MOSI · CS를 보며 비트 순서와 클록 주파수를 기록한다. (혼자 진행)

① 목적

SPI 통신이 실제 전선 위에서 어떻게 보이는지 확인한다. 우체부(마스터)가 박자(SCK)를 치면서 한 글자씩(MOSI) 불러 주고, 누구에게 말하는지는 손짓(CS)으로 알리는 것과 같다. 0x55 · 0xA5 · 0x00 · 0xFF를 보내 비트 순서가 MSB부터인지, SCK 주파수가 계산과 같은지 잰다.

② 이론 · 설계 계산

규칙	내용	이유
대기 상태	CS HIGH, SCK LOW	모드 0의 CPOL = 0
데이터 읽기	수신 측은 SCK **상승 에지**에서 MOSI를 읽는다	CPHA = 0 (첫 에지에서 읽음)
데이터 바꾸기	송신 측은 첫 비트를 첫 상승 에지 **전**에 내놓고, 이후 **하강 에지**에서 다음 비트로 바꾼다	읽는 순간(상승)과 바뀌는 순간(하강)이 반주기 떨어져 있어 MOSI가 안정된 상태에서 읽힌다
한 바이트	상승 에지 8번, MSB(b7)부터	수업 통일 규칙 · `SPI_FIRSTBIT_MSB`
프레임	CS가 LOW인 동안만 유효	여러 장치가 같은 SCK·MOSI를 공유해도 선택된 하나만 반응

수업 템플릿의 SPI1은 시스템 클록 100 MHz를 128로 나눈다.

$$f_{SCK} = \frac{100\ \text{MHz}}{128} = 781.25\ \text{kHz}, \qquad T_{SCK} = 1.28\ \mu\text{s}, \qquad T_{byte} = 8\,T_{SCK} = 10.24\ \mu\text{s}$$

CS는 SPI 하드웨어가 아니라 GPIO(PB6)로 코드가 직접 내린다. 그래서 CS LOW → 첫 SCK 사이에 `HAL_SPI_Transmit` 진입·SPI 켜기 시간(약 수 µs)이 들어가고, 마지막 SCK 뒤에도 함수가 끝날 때까지 CS가 LOW로 남는다. TB의 MCU 모델은 이 간격을 2 µs · 1.5 µs로 넣었다.

보내는 값	2진수 (b7→b0)	상승 에지에서 읽히는 MOSI	특징
0x55	0101 0101	0 1 0 1 0 1 0 1	MOSI가 SCK의 절반 속도로 계속 바뀜
0xA5	1010 0101	1 0 1 0 0 1 0 1	가운데 "0 0"에서 MOSI가 한 주기 동안 LOW 유지
0x00	0000 0000	0 0 0 0 0 0 0 0	MOSI는 계속 LOW, SCK만 8번
0xFF	1111 1111	1 1 1 1 1 1 1 1	MOSI는 CS LOW 동안 계속 HIGH

③ 포트 · 보드 매핑

신호	NUCLEO 핀	Arduino 이름	오실로스코프
SCK	PB3 · SPI1_SCK	D3	CH1 (트리거 아님)
MOSI	PA7 · SPI1_MOSI	D11	CH2
CS	PB6 · GPIO 출력	D10	CH1과 바꿔 보거나 다음 측정 (TBS2000B 2채널)
GND	CN6 GND	—	프로브 GND 클립

④ 핵심 코드

매뉴얼 송신 예제를 `polling_routine()` 에 그대로 넣었다(`LAB7/stm32/task8_tx`). 당일 다시 빌드하지 않으려고, 보내는 값만 1초마다 바꾸는 판(`task8_cycle`)을 하나 더 만들었다.

```
void polling_routine(void)
{
    /* LAB7 송신(마스터) · 매뉴얼 예제에서 보내는 값만 1초마다 0x55 -> 0xA5 -> 0x00 -> 0xFF로 바꿈.
     * 과제8(파형)·과제9·과제10에서 다시 빌드하지 않고 네 값을 모두 확인하기 위한 판. */
    static const uint8_t pattern[4] = {0x55U, 0xA5U, 0x00U, 0xFFU};
    static uint32_t last_ms = 0;
    static uint32_t k = 0;
    uint32_t now_ms = HAL_GetTick();
    if (now_ms - last_ms < 1000U) return;
    last_ms = now_ms;

    uint8_t tx = pattern[k];
    k = (k + 1U) & 3U;
```

```
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                      SPI_CS_ece2_Pin, GPIO_PIN_RESET);
    HAL_StatusTypeDef result =
        HAL_SPI_Transmit(&hspi1, &tx, 1U, 100U);
    HAL_GPIO_WritePin(SPI_CS_ece2_GPIO_Port,
                      SPI_CS_ece2_Pin, GPIO_PIN_SET);

    printf("TX=%02X, result=%d\r\n",
           (unsigned int)tx, (int)result);
}
```

⑤ 시뮬레이션 (PC 모의 실행)

LAB6에서 만든 모의 실행기를 SPI용으로 늘렸다. 가짜 `main.h`에 `SPI_HandleTypeDef` · `HAL_SPI_Transmit` · `HAL_GPIO_WritePin`을 넣고, 수업 코드 파일을 한 글자도 바꾸지 않고 gcc로 컴파일한다. 모의 HAL은 ① SPI1이 모드 0 · 8비트 · MSB인지 ② **CS(PB6)가 LOW일 때만 Transmit이 불렸는지**를 검사하고, 나간 바이트를 hex 파일로 남긴다. 이 파일이 과제9 수신 모의 실행과 과제10 FPGA TB의 입력이 된다.

매뉴얼 송신판 (4초)

```
# SPI1 init: MASTER, NSS=SOFT, CPOL=0 CPHA=0
(mode 0), 8-bit, MSB first, prescaler=128
# SCK = 100 MHz / 128 = 781.25 kHz
TX=55, result=0
TX=55, result=0
TX=55, result=0
TX=55, result=0
# PASS: 0 check failures
```

네 값 순환판 (8초)

```
# SPI1 init: MASTER, NSS=SOFT, CPOL=0 CPHA=0
(mode 0), 8-bit, MSB first, prescaler=128
# SCK = 100 MHz / 128 = 781.25 kHz
TX=55, result=0
TX=A5, result=0
TX=00, result=0
TX=FF, result=0
TX=55, result=0
TX=A5, result=0
TX=00, result=0
TX=FF, result=0
# PASS: 0 check failures
```

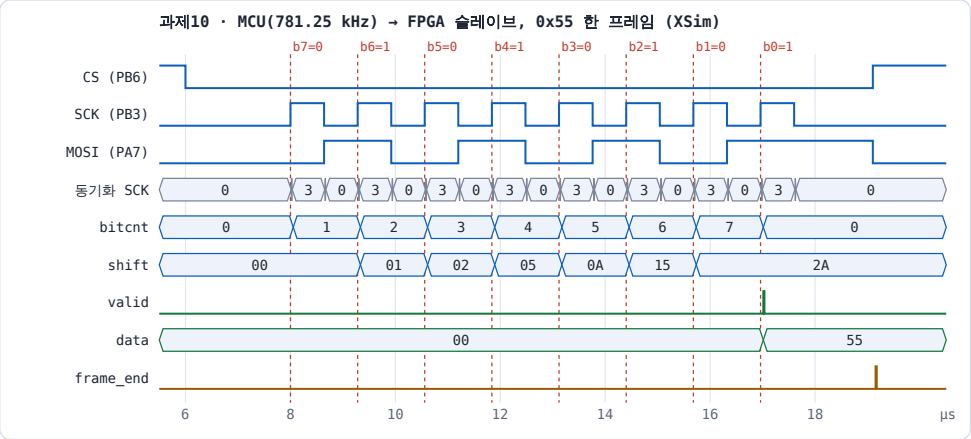


그림 1. MCU 송신 0x55의 예상 파형. 위 세 줄이 오실로스코프에 보일 신호(TB의 MCU 모델: 781.25 kHz, 모드 0, CS는 소프트웨어). 빨간 점선이 SCK 상승 에지 8번이며, 그때의 MOSI가 0 1 0 1 0 1 0 1이다. 아래 줄은 과제10에서 이 신호를 받은 FPGA 내부(XSim).

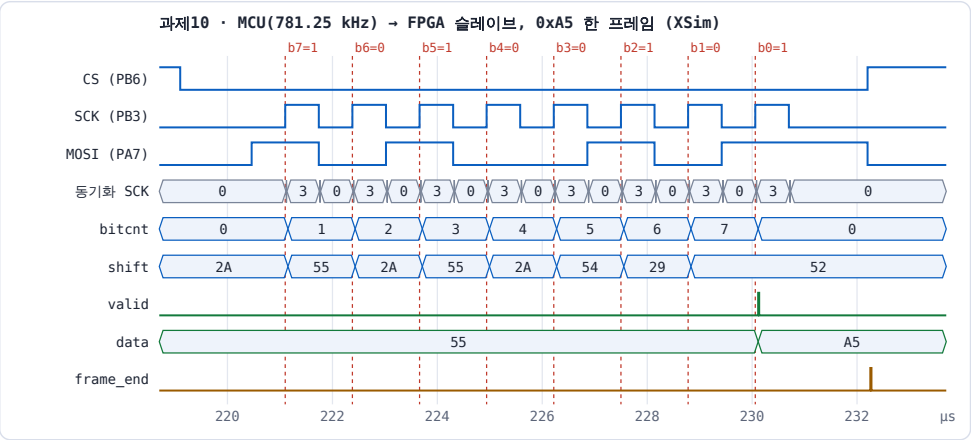


그림 2. 0xA5 = 1 0 1 0 0 1 0 1. 가운데 b3·b4가 연속 0이라 MOSI가 2.56 µs 동안 LOW로 머문다.

⑥ 빌드 결과

`mcu7 task8_tx`, `mcu7 task8_cycle` 모두 **BUILD OK**(STM32Cube CMake · arm-none-eabi-gcc, 경고 없음).

⑦ 보드 실습 계획

1. `mcu7 task8_cycle -Flash` → Serial Monitor(COM5, 9600)에 `TX=55, result=0` … 확인.

2. 오실로스코프: 3.3 V 신호라 **1 V/칸**. 한 프레임이 약 14 µs이므로 시간축은 **2 µs/칸**(20 µs 화면). 트리거는 CH1 하강 에지(CS를 볼 때) 또는 SCK 상승 에지, **Single**로 한 번 잡는다.
3. 값 네 개를 각각 찍고 MOSI를 상승 에지마다 읽어 표에 적는다. 커서로 SCK 주기를 재서 781 kHz와 비교한다.
4. 영상: 화면에 Serial Monitor의 TX 값과 오실로스코프 파형이 같이 보이게 찍는다.

2. 실험 2 · STM32_3 과제9 — MCU → MCU 수신 확인

C조와 D조의 MCU 두 대를 SPI로 연결해, 마스터가 보낸 값이 슬레이브의 VCP(PC 화면)에 같게 나오는지 확인하고 역할을 바꿔 한 번 더 한다.

① 목적

MCU 하나를 받는 쪽(슬레이브)으로 바꿔, SPI가 실제로 데이터를 전달하는지 받는 쪽 출력으로 확인한다. 과제8의 `result=0` 은 "보내기를 마쳤다"는 뜻일 뿐이라, 상대가 받았는지는 받는 쪽 화면으로만 알 수 있다.

② 이론 · 설계 계산

- 슬레이브는 SCK를 만들지 않고 받은 SCK로 비트를 읽는다. 매뉴얼 코드는 `ece2_474_init()` 에서 SPI1을 `HAL_SPI_DeInit` → `Mode = SPI_MODE_SLAVE` , `NSS = SPI_NSS_HARD_INPUT` → `HAL_SPI_Init` 으로 다시 설정한다. 나머지(모드 0 · 8비트 · MSB)는 마스터와 같은 설정을 그대로 쓴다.
- 장치 선택 입력은 **PA4 = SPI1 NSS**(AF5). 마스터의 CS 출력 PB6과 **핀 번호가 다르다**. NSS가 HIGH이면 슬레이브는 SCK를 무시한다.
- `HAL_SPI_Receive(&hspi1,&rx,1,2000)` 은 한 바이트가 올 때까지 최대 2초 기다린다. 마스터는 1초마다 보내므로 정상이면 매번 RX가 나오고, 마스터가 꺼져 있으면 2초마다 `RX timeout` 이 나온다. 그래서 **수신 측을 먼저 실행**한다.

③ 포트 · 보드 매핑 (역할 교대 포함)

신호	마스터 (송신)	슬레이브 (수신)	1차: C조→D조	2차: D조→C조
SCK	PB3	PB3	C PB3 → D PB3	D PB3 → C PB3
MOSI	PA7	PA7	C PA7 → D PA7	D PA7 → C PA7
CS / NSS	**PB6** (출력)	**PA4** (입력)	C PB6 → D PA4	**D PB6 → C PA4** (배선도 바꿈)
GND	GND	GND	공통	공통

④ 핵심 코드 (매뉴얼 수신 측 그대로)

```
void ece2_474_init(void)
{
    /* LAB7 수신 측 · 수업 매뉴얼 STM32_3 그대로: SPI1을 슬레이브(NSS 하드웨어 입력)로 다시 설정 */
    if (HAL_SPI_DeInit(&hspi1) != HAL_OK)
        Error_Handler();

    hspi1.Init.Mode = SPI_MODE_SLAVE;
    hspi1.Init.NSS = SPI_NSS_HARD_INPUT;
    if (HAL_SPI_Init(&hspi1) != HAL_OK)
        Error_Handler();

    GPIO_InitTypeDef gpio = {0};
    gpio.Pin = SPI_NSS_RX_ece2_Pin;
    gpio.Mode = GPIO_MODE_AF_PP;
    gpio.Pull = GPIO_PULLUP;
    gpio.Speed = GPIO_SPEED_FREQ_LOW;
    gpio.Alternate = GPIO_AF5_SPI1;
    HAL_GPIO_Init(SPI_NSS_RX_ece2_GPIO_Port, &gpio);
}
```

```
void polling_routine(void)
{
    /* LAB7 수신 측 · 수업 매뉴얼 STM32_3 그대로: 최대 2초 기다려 한 바이트 */
    uint8_t rx = 0;
    HAL_StatusTypeDef result =
        HAL_SPI_Receive(&hspi1, &rx, 1U, 2000U);
    if (result == HAL_OK)
    {
        printf("RX=%02X\r\n", (unsigned int)rx);
    }
    else if (result == HAL_TIMEOUT)
    {
        printf("RX timeout: check sender, SCK, MOSI, CS and GND\r\n");
    }
    else
    {
        printf("RX error=%d\r\n", (int)result);
    }
}
```

⑤ 시뮬레이션 (송신 모의 → 수신 모의)

과제8 순환판이 남긴 바이트 파일을 수신 코드의 `HAL_SPI_Receive` 가 차례로 돌려주게 했다. 모의 HAL은 init이 SPI1을 슬레이브 + NSS 하드웨어 입력으로 바꿨는지, PA4가 AF5 풀업인지 검사한다. 파일이 끝나면 송신 측이 멈춘 상황(2초 타임아웃)이 된다.

```
# SPI1 init: MASTER, NSS=SOFT, CPOL=0 CPHA=0 (mode 0), 8-bit, MSB first, prescaler=128
# SCK = 100 MHz / 128 = 781.25 kHz
# SPI1 init: SLAVE, NSS=HARD_INPUT, CPOL=0 CPHA=0 (mode 0), 8-bit, MSB first, prescaler=128
# PA4 init: mode=AF_PP pull=UP AF=5
RX=55
RX=A5
RX=00
RX=FF
RX=55
RX=A5
RX=00
RX=FF
RX timeout: check sender, SCK, MOSI, CS and GND
# PASS: 0 check failures
```

⑥ 빌드 결과

`mcu7 task9_rx` **BUILD OK**.

⑦ 보드 실습 계획

1. D조 MCU에 `task9_rx`, C조 MCU에 `task8_cycle`을 올린다. GND부터 연결한다.
2. 수신 측 Serial Monitor를 먼저 열고(`RX timeout` 이 2초마다 나오면 정상 대기), 송신 측 리셋.
3. D조 화면 RX = C조 화면 TX인지 4개 값 확인 → 영상.
4. 프로그램을 서로 바꾸고 **CS 선을 D조 PB6 → C조 PA4로 옮겨** 반복.

3. 실험 3 · STM32_3 과제10 — MCU → FPGA 수신 구현

자기 조 MCU(마스터)가 보낸 SPI 바이트를 FPGA 슬레이브 회로가 받아, FPGA UART(F6)로 PC에 "RX=55"처럼 출력해 MCU 송신값과 비교한다. FPGA 회로는 직접 작성.

① 목적

FPGA에 SPI 받는 회로를 만든다. FPGA는 자기 50 MHz 박자로 움직이는데, MCU가 보내는 SCK는 FPGA와 무관한 박자다. 남이 치는 북소리를 내 시계로 매우 자주 확인하면서 "방금 북이 울렸다"를 알아차리는 방식으로 받는다.

② 이론 · 설계 계산

(1) SCK를 클록으로 쓰지 않는 이유

SCK를 그대로 플립플롭 클록에 넣으면 FPGA 안에 클록이 두 개가 되고, 받은 값을 50 MHz 쪽 UART로 옮길 때 또 동기화가 필요하다. 대신 SCK · MOSI · CS를 모두 50 MHz로 **샘플링**하고 상승 에지를 "앞 클록 0, 지금 1"로 찾는다. 이렇게 하면 회로 전체가 한 클록으로 움직인다.

(2) 샘플링이 충분히 빠른가

$$\frac{T_{SCK}}{2} = 640\ \text{ns} = 32\,T_{clk}\ (20\ \text{ns})$$

SCK의 HIGH · LOW가 각각 32클록 유지되므로 에지를 놓칠 수 없다. 2단 동기화로 생기는 지연은 40~60 ns(2~3클록)이다. SCK와 MOSI가 **같은 2단**을 거치므로 둘의 시간 관계가 그대로 유지된다. MOSI는 하강 에지에서 바뀌어 상승 에지까지 640 ns 동안 안정하므로, 동기화된 상승 에지에서 읽는 MOSI는 항상 올바른 비트다. 이론상 HIGH·LOW가 각각 3클록 이상이면 되므로 약 8 MHz까지 받을 수 있다(수업 조건 1 MHz 이하의 8배 여유).

(3) 동기화가 필요한 이유

외부 신호가 플립플롭의 셋업·홀드 시간 안에 바뀌면 출력이 잠시 0도 1도 아닌 상태(준안정)가 될 수 있다. 첫 플립플롭이 흔들려도 한 클록(20 ns) 동안 안정될 시간을 주고 두 번째 플립플롭에서 다시 읽으면 그 확률이 무시할 만큼 작아진다. 두 플립플롭에 `ASYNC_REG` 속성을 붙여 Vivado가 서로 가까이 배치하게 했다.

(4) 수신 상태 규칙

조건 (동기화 후)	동작
CS = HIGH	bitcnt = 0, 바이트 번호 = 0. 방금 HIGH가 됐으면 `frame_end`, 비트가 덜 찼으면 `frame_abort` 도 1클록
CS가 막 LOW가 됨	`frame_start`, bitcnt = 0
CS = LOW, SCK 상승	shift ← {shift, MOSI}. bitcnt = 7이면 `data = {shift, MOSI}`, `valid` 1클록, bitcnt = 0, 바이트 번호 + 1
CS = LOW, 그 밖	유지

(5) UART 출력 시간

$$T_{line} = \frac{7\ \text{글자} \times 10\ \text{비트}}{9600.6\ \text{bps}} = 7.29\ \text{ms}\ \ (\texttt{"RX=55\textbackslash r\textbackslash n"})$$

MCU는 1초마다 보내므로 UART는 0.7%만 바쁘다. 그래도 프레임이 몰려 오는 경우(4바이트 프레임은 16 글자 = 16.7 ms)를 위해 64칸 FIFO를 두었다.

③ 포트 · 보드 매핑

포트	방향	FPGA 핀	연결
clk_50mhz	입력	B6	보드 클록 (CLOCK SELECT = 50 MHz)
rst_p	입력	K4	키패드 1 (누르면 리셋)
spi_sck	입력	**F8** (BDIO I/O 00, J10)	← MCU PB3
spi_mosi	입력	**E8** (I/O 01)	← MCU PA7
spi_cs_n	입력	**A7** (I/O 02)	← MCU PB6
uart_txd	출력	F6	보드 USB-UART(FTDI) → PC COM6
led[7:0]	출력	L4 M4 M2 N7 M7 M3 M1 N5	LED1 = b7 … LED8 = b0
GND	—	J10 아래쪽 GND	↔ MCU GND

CS 입력에는 내부 풀업, SCK에는 내부 풀다운을 걸었다. 선이 빠져도 "선택 안 됨 · 대기 LOW"로 머물러 잡음 바이트가 생기지 않는다(LAB6에서 PORT ENABLE ON으로 핀이 떠서 잡음을 받은 경험). PORT A ENABLE(DIP SW5-2)은 **OFF**여야 핀이 연결된다.

④ RTL

spi_slave_rx.v — SPI 수신기

```
`timescale 1ns/1ps
// LAB7 과제10 · SPI 슬레이브 수신기 (모드 0, MSB 먼저, 8비트 단위, MISO 없음)
//
//  - 외부 SCK·MOSI·CS는 FPGA 클록(50 MHz)과 무관하게 바뀌므로 각각 2단 플립플롭으로 동기화한다.
//  - 동기화된 SCK가 0 -> 1로 바뀐 클록(상승 에지)에서 동기화된 MOSI를 시프트 레지스터에 넣는다.
//    SCK와 MOSI가 같은 2단을 거치므로 둘 사이의 시간 관계(MOSI는 하강 에지에서 바뀜)가 그대로 유지된다.
//  - 비트 카운터가 8을 채우면 data/valid(1클록)를 내고 카운터를 0으로 돌려 같은 CS 구간의 다음 바이트도 받는다.
//    (기말 프로젝트의 4바이트 레지스터 맵처럼 한 프레임에 여러 바이트를 보내도 그대로 쓸 수 있다.)
//  - CS가 HIGH가 되면 프레임이 끝난 것: 비트 카운터를 0으로(덜 찬 비트는 버림) 하고 frame_end를 낸다.
//    덜 찬 비트가 있었으면 frame_abort도 함께 낸다.
//  - 50 MHz에서 MCU SCK 781.25 kHz의 반주기 640 ns = 32클록이므로 매 에지를 놓치지 않는다.
module spi_slave_rx (
    input  wire       clk,
    input  wire       rst_p,
    input  wire       sck_in,       // 마스터가 만드는 클록 (대기 LOW)
    input  wire       mosi_in,      // 마스터 -> 슬레이브 데이터
    input  wire       cs_n_in,      // LOW일 때 통신
    output reg  [7:0] data,         // 방금 받은 바이트
    output reg        valid,        // data가 새로 들어온 클록에 1
    output reg  [7:0] byte_idx,     // 이 프레임에서 몇 번째 바이트인지 (0부터)
    output reg        frame_start,  // CS 하강 (1클록)
    output reg        frame_end,    // CS 상승 (1클록)
    output reg        frame_abort,  // CS 상승 때 8비트가 덜 찬 비트가 남아 있었음 (1클록)
    output wire       cs_active     // 동기화된 CS가 LOW
);
    (* ASYNC_REG = "TRUE" *) reg [1:0] sck_s;
    (* ASYNC_REG = "TRUE" *) reg [1:0] mosi_s;
    (* ASYNC_REG = "TRUE" *) reg [1:0] cs_s;
    reg       sck_d, cs_d;          // 한 클록 전 값 (에지 검출)
    reg [6:0] shift;                // 앞서 받은 7비트
    reg [2:0] bitcnt;               // 이 바이트에서 받은 비트 수 0~7
    reg [7:0] count;                // 이 프레임에서 완성된 바이트 수

    wire sck = sck_s[1], mosi = mosi_s[1], cs_n = cs_s[1];
    wire sck_rise = sck & ~sck_d;
    assign cs_active = ~cs_n;

    always @(posedge clk) begin
        if (rst_p) begin
            sck_s <= 2'b00; mosi_s <= 2'b00; cs_s <= 2'b11;
            sck_d <= 1'b0;  cs_d <= 1'b1;
            shift <= 7'd0;  bitcnt <= 3'd0; count <= 8'd0;
            data <= 8'd0;   valid <= 1'b0;  byte_idx <= 8'd0;
            frame_start <= 1'b0; frame_end <= 1'b0; frame_abort <= 1'b0;
        end else begin
            // 1) 2단 동기화와 한 클록 전 값
            sck_s  <= {sck_s[0],  sck_in};
            mosi_s <= {mosi_s[0], mosi_in};
            cs_s   <= {cs_s[0],   cs_n_in};
            sck_d  <= sck;
            cs_d   <= cs_n;
            // 2) 펄스 출력은 기본 0
            valid <= 1'b0; frame_start <= 1'b0; frame_end <= 1'b0; frame_abort <= 1'b0;

            if (cs_n) begin
                // CS HIGH: 통신 없음. 프레임이 막 끝났으면 알리고 덜 찬 비트는 버린다.
                if (!cs_d) begin
                    frame_end   <= 1'b1;
                    frame_abort <= (bitcnt != 3'd0);
                end
                bitcnt <= 3'd0;
                count  <= 8'd0;
            end else if (cs_d) begin
                // CS가 막 LOW가 됨: 새 프레임 시작
                frame_start <= 1'b1;
                bitcnt <= 3'd0;
                count  <= 8'd0;
            end else if (sck_rise) begin
                // SCK 상승 에지: MOSI 한 비트 읽기 (MSB 먼저 -> 왼쪽으로 민다)
                if (bitcnt == 3'd7) begin
                    data     <= {shift, mosi};
                    valid    <= 1'b1;
                    byte_idx <= count;
                    count    <= count + 8'd1;
                    bitcnt   <= 3'd0;
                end else begin
                    shift  <= {shift[5:0], mosi};
                    bitcnt <= bitcnt + 3'd1;
                end
            end
        end
    end
endmodule
```

spi_uart_log.v — 받은 바이트를 글자 줄로 바꿔 UART로 출력

```
`timescale 1ns/1ps
// LAB7 공용 · SPI 바이트를 사람이 읽는 글자 한 줄로 바꿔 FPGA USB-UART(F6)로 보내는 기록기
//
//  한 프레임(CS LOW 구간) = 한 줄.  예) 한 바이트 0x55  -> "RX=55\r\n"   (MCU 수신 예제의 출력과 같은 모양)
//                                   네 바이트         -> "RX=12 34 56 78\r\n"
//                                   8비트가 덜 찬 채 CS 해제 -> 줄 끝에 " ?" (예: "RX=?\r\n")
//  0x00·0xFF·0xA5처럼 화면에 안 보이는 값도 16진수 글자로 보이므로 Serial Monitor에서 바로 비교할 수 있다.
//
//  SPI 한 바이트는 10~16 us, UART 한 글자는 1.04 ms라 UART가 훨씬 느리다.
//  그래서 받은 것(토큰)을 FIFO에 먼저 쌓고, 출력기는 UART가 빌 때마다 하나씩 꺼내 글자로 바꾼다.
//    토큰 = {종류[1:0], 바이트[7:0]}   종류 0: 바이트, 1: 프레임 끝, 2: 덜 찬 채 끝
module spi_uart_log #(
    parameter integer DIV      = 5208,      // 50 MHz / 9600
    parameter [7:0]   P0       = "R",       // 줄 머리 두 글자 ("RX" 또는 "TX")
    parameter [7:0]   P1       = "X",
    parameter integer AW       = 6          // FIFO 깊이 2^AW = 64 토큰
)(
    input  wire       clk,
    input  wire       rst_p,
    input  wire       byte_valid,
    input  wire [7:0] byte_data,
    input  wire       frame_end,
    input  wire       frame_abort,
    output wire       uart_txd,
    output reg        overflow          // FIFO가 넘친 적이 있으면 1 (리셋까지 유지)
);
    // ---------- 토큰 FIFO ----------
    reg [9:0] mem [0:(1<<AW)-1];
    reg [AW:0] wp, rp;
    wire empty = (wp == rp);
    wire full  = (wp[AW] != rp[AW]) && (wp[AW-1:0] == rp[AW-1:0]);
    // 같은 클록에 바이트와 프레임 끝이 함께 올 수는 없다(바이트 완성은 SCK 에지, 끝은 CS 상승).
    wire       push = byte_valid | frame_end;
    wire [9:0] tok  = byte_valid ? {2'd0, byte_data} : {(frame_abort ? 2'd2 : 2'd1), 8'd0};
    reg        pop;

    always @(posedge clk) begin
        if (rst_p) begin wp <= 0; rp <= 0; overflow <= 1'b0; end
        else begin
            if (push) begin
                if (!full) begin mem[wp[AW-1:0]] <= tok; wp <= wp + 1'b1; end
                else overflow <= 1'b1;
            end
            if (pop) rp <= rp + 1'b1;
        end
    end

    // ---------- 16진수 한 자리 -> ASCII ----------
    function [7:0] hexc(input [3:0] n);
        hexc = (n < 4'd10) ? (8'h30 + {4'd0, n}) : (8'h41 + {4'd0, n} - 8'd10);
    endfunction

    // ---------- 출력기: 토큰 하나를 최대 6글자로 바꿔 차례로 UART에 넘김 ----------
    reg  [7:0] buf0, buf1, buf2, buf3, buf4, buf5;
    reg  [2:0] blen, bpos;
    reg        line_open;                  // 이번 줄에 "RX=" 를 이미 썼는가
    reg        tx_valid; reg [7:0] tx_data;
    wire       tx_ready;
    wire [9:0] head = mem[rp[AW-1:0]];

    function [7:0] pick(input [2:0] i, input [7:0] a, b, c, d, e, f);
        case (i) 3'd0: pick = a; 3'd1: pick = b; 3'd2: pick = c; 3'd3: pick = d; 3'd4: pick = e; default:
pick = f; endcase
    endfunction

    always @(posedge clk) begin
        if (rst_p) begin
            blen <= 0; bpos <= 0; line_open <= 1'b0; pop <= 1'b0;
            tx_valid <= 1'b0; tx_data <= 8'h00;
            {buf0, buf1, buf2, buf3, buf4, buf5} <= 48'd0;
        end else begin
            pop <= 1'b0; tx_valid <= 1'b0;
            if (bpos != blen) begin
                // 글자 보내는 중: UART가 비면 다음 글자
                if (tx_ready && !tx_valid) begin
                    tx_data  <= pick(bpos, buf0, buf1, buf2, buf3, buf4, buf5);
                    tx_valid <= 1'b1;
                    bpos     <= bpos + 3'd1;
                end
            end else if (!empty && !pop) begin
                // 다음 토큰을 글자로 바꾼다
                pop <= 1'b1; bpos <= 3'd0;
                case (head[9:8])
                    2'd0: begin                              // 바이트
                        if (!line_open) begin {buf0,buf1,buf2,buf3,buf4} <= {P0, P1, "=", hexc(head[7:4]),
hexc(head[3:0])}; blen <= 3'd5; end
                        else            begin {buf0,buf1,buf2}           <= {" ",         hexc(head[7:4]),
hexc(head[3:0])}; blen <= 3'd3; end
                        line_open <= 1'b1;
                    end
                    2'd1: begin                              // 정상 끝
                        if (line_open) begin {buf0,buf1} <= {8'h0D, 8'h0A}; blen <= 3'd2; end
```

```
                        else blen <= 3'd0;                  // 바이트 없이 CS만 내려갔다 올라감: 출력 없음
                        line_open <= 1'b0;
                    end
                    default: begin                          // 덜 찬 비트를 남기고 끝
                        if (line_open) begin {buf0,buf1,buf2,buf3}          <= {" ", "?", 8'h0D, 8'h0A};
blen <= 3'd4; end
                        else           begin {buf0,buf1,buf2,buf3,buf4,buf5} <= {P0, P1, "=", "?", 8'h0D,
8'h0A}; blen <= 3'd6; end
                        line_open <= 1'b0;
                    end
                endcase
            end
        end
    end

    uart_tx #(.DIV(DIV)) u_tx (.clk(clk), .rst_p(rst_p), .valid(tx_valid), .data(tx_data),
                               .ready(tx_ready), .tx(uart_txd));
endmodule
```

spi_rx_top.v — 보드 최상위

```
`timescale 1ns/1ps
// LAB7 과제10 · MCU(SPI 마스터) -> FPGA(SPI 슬레이브) 수신, 받은 값을 PC로 출력
//
//  spi_sck / spi_mosi / spi_cs_n : MCU PB3 / PA7 / PB6에서 오는 신호. 확장 포트 BDIO(PORT A)로 받는다.
//  uart_txd : 받은 바이트를 "RX=55\r\n" 모양 글자로 보드 USB-UART(F6)를 통해 PC에 출력
//  led      : 마지막으로 받은 바이트 (LED1 = 최상위 비트 b7 ... LED8 = b0, 1이면 켜짐)
//  같은 회로를 과제12(FPGA -> FPGA)의 수신 쪽으로도 그대로 쓴다.
module spi_rx_top #(
    parameter integer CLK_HZ = 50_000_000,
    parameter integer BAUD   = 9_600
)(
    input  wire       clk_50mhz,
    input  wire       rst_p,          // 키패드 1 (누르면 HIGH)
    input  wire       spi_sck,
    input  wire       spi_mosi,
    input  wire       spi_cs_n,
    output wire       uart_txd,
    output wire [7:0] led
);
    localparam integer DIV = (CLK_HZ + BAUD/2) / BAUD;   // 5208 클록/비트 (9600.6 bps)

    wire [7:0] rx_data, rx_idx;
    wire rx_valid, f_start, f_end, f_abort, cs_on, ovf;
    reg  [7:0] last;

    spi_slave_rx u_spi (
        .clk(clk_50mhz), .rst_p(rst_p),
        .sck_in(spi_sck), .mosi_in(spi_mosi), .cs_n_in(spi_cs_n),
        .data(rx_data), .valid(rx_valid), .byte_idx(rx_idx),
        .frame_start(f_start), .frame_end(f_end), .frame_abort(f_abort), .cs_active(cs_on));

    spi_uart_log #(.DIV(DIV), .P0("R"), .P1("X")) u_log (
        .clk(clk_50mhz), .rst_p(rst_p),
        .byte_valid(rx_valid), .byte_data(rx_data),
        .frame_end(f_end), .frame_abort(f_abort),
        .uart_txd(uart_txd), .overflow(ovf));

    always @(posedge clk_50mhz)
        if (rst_p) last <= 8'h00;
        else if (rx_valid) last <= rx_data;

    assign led = last;
endmodule
```

`uart_tx` 는 LAB3-07 · LAB6 과제6에서 보드로 검증한 `uart.v` 를 그대로 쓴다(9600.6 bps).

⑤ 시뮬레이션

TB(`LAB7-01/sim/tb.sv`)는 MCU의 SPI1 마스터를 파형으로 흉내 낸다(781.25 kHz, 모드 0, CS LOW → 첫 상승 2 µs, 끝 → CS HIGH 1.5 µs). 보내는 바이트는 **과제8 MCU 모의 실행이 남긴 파일**(55 A5 00 FF ×2)을 그대로 읽는다. FPGA 클록 위상은 3.7 ns 어긋나게 두었다. PC 쪽 UART 수신 모델이 F6의 글자를 줄 단위로 모아 기대값과 비교한다.

#	시나리오	확인할 것
1	MCU 모의 실행 바이트 8개, 1프레임 1바이트	수신 바이트 · 순서 · 개수, PC 줄 "RX=55" ...
2	3비트만 보내고 CS 해제	덜 찬 비트 버림 → "RX=?", 바로 다음 0x3C 정상
3	CS LOW 그대로 4바이트 12 34 56 78	바이트 번호 0~3, 한 줄 "RX=12 34 56 78"
4	CS HIGH인 동안 SCK 펄스 20개	무시 (다른 장치 통신·잡음)
5	SCK 1 MHz와 100 kHz	속도와 무관하게 정상
끝	UART 출력이 다 나올 때까지	FIFO 넘침 없음, LED = 마지막 값 0x7E

XSim 결과 (`results/xsim_result.txt` , Icarus와 숫자가 모두 같음)

```
MCU 모의 실행이 보낸 바이트 8개 (sim/spi1_tx_task8_cycle.hex)
17.034 us  FPGA SPI 수신 #0 = 0x55 (프레임 안 0번째)   LED=01010101
230.134 us  FPGA SPI 수신 #1 = 0xa5 (프레임 안 0번째)   LED=10100101
443.234 us  FPGA SPI 수신 #2 = 0x00 (프레임 안 0번째)   LED=00000000
656.334 us  FPGA SPI 수신 #3 = 0xff (프레임 안 0번째)   LED=11111111
869.434 us  FPGA SPI 수신 #4 = 0x55 (프레임 안 0번째)   LED=01010101
1082.534 us  FPGA SPI 수신 #5 = 0xa5 (프레임 안 0번째)   LED=10100101
1295.634 us  FPGA SPI 수신 #6 = 0x00 (프레임 안 0번째)   LED=00000000
1508.734 us  FPGA SPI 수신 #7 = 0xff (프레임 안 0번째)   LED=11111111
1748.534 us  FPGA SPI 수신 #8 = 0x3c (프레임 안 0번째)   LED=00111100
1811.634 us  FPGA SPI 수신 #9 = 0x12 (프레임 안 0번째)   LED=00010010
1821.874 us  FPGA SPI 수신 #10 = 0x34 (프레임 안 1번째)   LED=00110100
1832.114 us  FPGA SPI 수신 #11 = 0x56 (프레임 안 2번째)   LED=01010110
1842.354 us  FPGA SPI 수신 #12 = 0x78 (프레임 안 3번째)   LED=01111000
1927.454 us  FPGA SPI 수신 #13 = 0xc3 (프레임 안 0번째)   LED=11000011
1988.454 us  FPGA SPI 수신 #14 = 0x81 (프레임 안 0번째)   LED=10000001
2116.814 us  FPGA SPI 수신 #15 = 0x7e (프레임 안 0번째)   LED=01111110
7256.455 us  PC(FPGA COM) <- "RX=55"
14547.935 us  PC(FPGA COM) <- "RX=A5"
21839.415 us  PC(FPGA COM) <- "RX=00"
29130.895 us  PC(FPGA COM) <- "RX=FF"
36422.375 us  PC(FPGA COM) <- "RX=55"
43713.855 us  PC(FPGA COM) <- "RX=A5"
51005.335 us  PC(FPGA COM) <- "RX=00"
58296.815 us  PC(FPGA COM) <- "RX=FF"
64546.655 us  PC(FPGA COM) <- "RX=?"
71838.135 us  PC(FPGA COM) <- "RX=3C"
88504.375 us  PC(FPGA COM) <- "RX=12 34 56 78"
95795.855 us  PC(FPGA COM) <- "RX=C3"
103087.335 us  PC(FPGA COM) <- "RX=81"
110378.815 us  PC(FPGA COM) <- "RX=7E"
PASS spi_rx_top: 16 bytes, 14 lines (partial frame discarded, 4-byte frame, SCK noise with CS high
ignored, 1 MHz / 100 kHz ok)
```

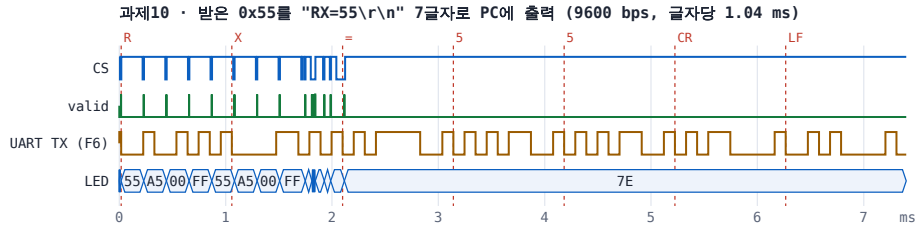


그림 3. 받은 0x55를 F6로 "R X = 5 5 CR LF" 7글자로 보내는 데 7.29 ms가 걸린다. 그 사이 TB가 보낸 다음 프레임들(A5 · 00 · FF …, CS 펄스)은 FIFO에 쌓였다가 차례로 출력된다(LED는 마지막 수신값).

그림 1 · 2의 아래 줄이 수신 회로 내부다. 동기화된 SCK(2비트 시프트 값 3 = "방금 두 번 연속 1")가 바뀐 클록마다 bitcnt가 1씩 늘고, shift에 01 → 02 → 05 → 0A → 15 → 2A처럼 비트가 왼쪽에서 들어온다. 8번째 상승 에지에서 data = 55, valid가 1클록 나온다. frame_end는 CS가 올라간 뒤 동기화 지연만큼 늦게 나온다.

⑥ 합성 · 구현 결과 (보드 핀 XDC, `build7 1`)

항목	값	설명
Slice LUTs	160 / 48,000 (0.33%)	FIFO 64×10은 LUT RAM
Slice Registers	141	
Bonded IOB	14	클록 1 + 리셋 1 + SPI 3 + UART 1 + LED 8
WNS / WHS	**+14.907 ns** / +0.036 ns	50 MHz(20 ns) 셋업·홀드 모두 만족
비트스트림	`LAB7_bit/LAB7-01_spi_rx.bit`	실험 전에 미리 생성

⑦ 보드 실습 계획

1. FPGA: CLOCK SELECT 50 MHz(F), PORT A ENABLE OFF 확인 → `prog7 1`. 노트북 USB가 2개라 비트스트림을 올린 뒤 JTAG을 빼고 FTDI를 꽂는다(COM6).
2. MCU: `mcu7 task8_cycle -Flash`. 배선 PB3 → F8, PA7 → E8, PB6 → A7, GND ↔ GND.
3. Serial Monitor 두 개: MCU(COM5) `TX=55, result=0` · FPGA(COM6) `RX=55`가 1초마다 같은 값인지, LED가 01010101인지 확인.
4. 안 나오면 순서대로: COM6 존재(FTDI Load VCP) → MCU 파형(과제8) → CLOCK SELECT → PORT ENABLE OFF → 핀·GND. "RX=?"가 나오면 SCK 선 접촉 불량을 의심한다.

4. 실험 4 · STM32_3 과제11 — FPGA → MCU 송신 구현

FPGA를 SPI 마스터로 만들어 1 MHz 이하의 SCK로 1초마다 한 바이트를 보내고, 오실로스코프로 파형을 본 뒤 슬레이브 MCU의 VCP 출력과 비교한다. FPGA 회로는 직접 작성.

① 목적

이번에는 FPGA가 박자를 치는 쪽이다. SCK · MOSI · CS를 모두 FPGA가 만들고, 모드 0 규칙(첫 비트는 미리, 나머지는 하강 에지에서)을 정확히 지키는지 파형 단위로 검사한다.

② 이론 · 설계 계산

$$f_{SCK} = \frac{f_{clk}}{2\,N_{half}} = \frac{50\ \text{MHz}}{2 \times 50} = 500\ \text{kHz}\ \le 1\ \text{MHz}$$

반주기 50클록 = 1 µs. MCU 슬레이브는 SPI 클록이 PCLK/4 이하이면 받을 수 있어 500 kHz는 충분히 느리다. 한 프레임의 CS LOW 시간은

$$T_{CS} = \underbrace{1}_{\text{CS}\to\text{첫 상승}} + \underbrace{7.5 \times 2}_{\text{첫 상승}\to\text{마지막 하강}} + \underbrace{1}_{\text{마지막 하강}\to\text{CS}} = 17\ \mu s$$

상태	하는 일	머무는 시간
IDLE	start가 오면 CS ↓, **MOSI = b7**(첫 상승 전에 미리), left = 8	—
SETUP	시간이 차면 SCK ↑ (슬레이브가 b7을 읽음)	1 µs
HIGH	SCK ↓. 남은 비트가 있으면 **이 순간 MOSI를 다음 비트로**, 마지막이면 HOLD로	1 µs
LOW	SCK ↑ (다음 비트 읽힘)	1 µs
HOLD	CS ↑, MOSI = 0, done	1 µs
GAP	CS HIGH 최소 유지 후 IDLE	1 µs

SCK · MOSI · CS가 모두 플립플롭 출력에서 바로 나가므로 조합 논리 글리치가 없다. 보낼 값은 DIP SW1~SW8이며 2단 동기화 후 송신 시작 순간에 붙잡는다(전송 중 스위치를 움직여도 그 프레임은 바뀌지 않음).

③ 포트 · 보드 매핑

포트	방향	FPGA 핀	연결
spi_sck	출력	**F8** (I/O 00)	→ MCU PB3
spi_mosi	출력	**E8** (I/O 01)	→ MCU PA7
spi_cs_n	출력	**A7** (I/O 02)	→ MCU **PA4** (NSS)
sw[7:0]	입력	Y1 W3 U2 T1 W4 W1 V4 U4	SW1 = b7 … SW8 = b0, ON = 1
uart_txd	출력	F6	보낸 값 "TX=55" 기록
led[7:0]	출력	과제10과 같음	마지막으로 보낸 값
clk · rst	입력	B6 · K4	

과제10과 **같은 세 핀**을 쓰되 방향만 반대다. 그래서 과제12에서는 두 보드의 F8 · E8 · A7끼리 그대로 잇는다. 과제10 비트스트림이 올라간 상태에서 MCU 마스터 출력을 연결하는 것은 문제없지만, 과제11 비트스트림(FPGA 출력)에 MCU **출력**을 연결하면 두 출력이 맞붙으므로 MCU에는 반드시 수신 코드(`task9_rx`)를 먼저 올린다.

④ RTL

spi_master_tx.v — SPI 송신기

```
`timescale 1ns/1ps
// LAB7 과제11 · SPI 마스터 송신기 (모드 0, MSB 먼저, MISO 없음)
//
//  start 펄스가 오면 tx_data(NBYTES 바이트, 위쪽 바이트부터)를 한 프레임으로 보낸다.
//    1) CS를 LOW로 내리면서 MOSI에 첫 비트(최상위 비트)를 미리 놓는다.        -> "첫 상승 에지 전에 MSB 준비"
//    2) CS_SETUP 클록 뒤 SCK를 올린다 (상승 에지: 슬레이브가 MOSI를 읽는 순간).
//    3) HALF_DIV 클록 뒤 SCK를 내리면서 MOSI를 다음 비트로 바꾼다.             -> "하강 에지에서 다음 비트"
//    4) 마지막 비트의 하강 에지 뒤 CS_HOLD 클록 기다렸다가 CS를 HIGH로 올린다.
//    5) CS HIGH를 HALF_DIV 클록 이상 유지한 뒤 다음 start를 받는다.
//  SCK 주파수 = CLK / (2 x HALF_DIV).  50 MHz, HALF_DIV = 50 -> 500 kHz (조건: 1 MHz 이하)
//  모든 출력은 플립플롭에서 바로 나가므로 글리치가 없다.
module spi_master_tx #(
```

```
    parameter integer HALF_DIV = 50,          // SCK 반주기 클록 수
    parameter integer NBYTES   = 1,           // 한 프레임 바이트 수 (기말 프로젝트: 4)
    parameter integer CS_SETUP = HALF_DIV,    // CS 하강 -> 첫 SCK 상승
    parameter integer CS_HOLD  = HALF_DIV     // 마지막 SCK 하강 -> CS 상승
)(
    input  wire                 clk,
    input  wire                 rst_p,
    input  wire                 start,        // IDLE일 때 1클록 펄스
    input  wire [8*NBYTES-1:0]  tx_data,
    output wire                 busy,
    output reg                  done,         // 프레임이 끝나 CS가 올라간 클록에 1
    output reg                  sck,
    output reg                  mosi,
    output reg                  cs_n
);
    localparam integer NBITS = 8 * NBYTES;
    localparam [2:0] S_IDLE = 3'd0, S_SETUP = 3'd1, S_HIGH = 3'd2, S_LOW = 3'd3, S_HOLD = 3'd4, S_GAP =
3'd5;

    reg [2:0]          state;
    reg [NBITS-1:0]    shreg;                 // 맨 위 비트가 지금 MOSI에 나가 있는 비트
    reg [15:0]         left;                  // 남은 비트 수 (지금 나가 있는 비트 포함)
    reg [15:0]         timer;

    assign busy = (state != S_IDLE);

    always @(posedge clk) begin
        if (rst_p) begin
            state <= S_IDLE; shreg <= {NBITS{1'b0}}; left <= 16'd0; timer <= 16'd0;
            sck <= 1'b0; mosi <= 1'b0; cs_n <= 1'b1; done <= 1'b0;
        end else begin
            done <= 1'b0;
            if (timer != 16'd0) timer <= timer - 16'd1;
            else case (state)
                S_IDLE: if (start) begin
                    shreg <= tx_data;
                    mosi  <= tx_data[NBITS-1];           // MSB를 미리 놓는다
                    cs_n  <= 1'b0;
                    left  <= NBITS;
                    timer <= CS_SETUP - 1;
                    state <= S_SETUP;
                end
                S_SETUP, S_LOW: begin                    // SCK 상승
                    sck   <= 1'b1;
                    timer <= HALF_DIV - 1;
                    state <= S_HIGH;
                end
                S_HIGH: begin                            // SCK 하강
                    sck <= 1'b0;
                    if (left == 16'd1) begin             // 방금 마지막 비트를 읽혔다
                        timer <= CS_HOLD - 1;
                        state <= S_HOLD;
                    end else begin                       // 다음 비트로 바꾼다
                        shreg <= {shreg[NBITS-2:0], 1'b0};
                        mosi  <= shreg[NBITS-2];
                        left  <= left - 16'd1;
                        timer <= HALF_DIV - 1;
                        state <= S_LOW;
                    end
                end
                S_HOLD: begin                            // CS 해제
                    cs_n  <= 1'b1;
                    mosi  <= 1'b0;
                    done  <= 1'b1;
                    timer <= HALF_DIV - 1;
                    state <= S_GAP;
                end
                default: state <= S_IDLE;                // S_GAP: CS HIGH 최소 유지 시간 끝
            endcase
        end
    end
endmodule
```

spi_tx_top.v — 보드 최상위

```
`timescale 1ns/1ps
// LAB7 과제11 · FPGA(SPI 마스터) -> MCU(SPI 슬레이브) 송신, 1초마다 한 바이트
//
//  보낼 값 = DIP 스위치 SW1~SW8 (SW1 = 최상위 비트 b7, ON = 1).  예) 0x55 = SW2·SW4·SW6·SW8 ON
//    -> 당일 다시 빌드하지 않고 0x55 · 0xA5 · 0x00 · 0xFF를 바꿔 가며 확인할 수 있다.
//  spi_sck / spi_mosi / spi_cs_n : 확장 포트 BDIO(PORT A)로 내보내 MCU PB3 / PA7 / PA4(NSS)에 연결
//  SCK = 50 MHz / (2 x 50) = 500 kHz
//  uart_txd : 보낸 값을 "TX=55\r\n" 모양으로 PC에 출력 (과제12에서 송신 쪽 기록용)
//  led      : 마지막으로 보낸 바이트
//  같은 회로를 과제12(FPGA -> FPGA)의 송신 쪽으로도 그대로 쓴다.
module spi_tx_top #(
    parameter integer CLK_HZ   = 50_000_000,
    parameter integer BAUD     = 9_600,
    parameter integer PERIOD   = 50_000_000    // 송신 간격(클록 수) = 1초. TB에서만 줄여 쓴다.
)(
    input  wire       clk_50mhz,
    input  wire       rst_p,
    input  wire [7:0] sw,
```

```
    output wire       spi_sck,
    output wire       spi_mosi,
    output wire       spi_cs_n,
    output wire       uart_txd,
    output wire [7:0] led
);
    localparam integer DIV  = (CLK_HZ + BAUD/2) / BAUD;
    localparam integer HALF = CLK_HZ / 1_000_000;            // 50 MHz -> 50클록 = 1 us 반주기 -> 500 kHz

    // DIP 스위치 동기화 (기계식 입력)
    (* ASYNC_REG = "TRUE" *) reg [7:0] sw_s0, sw_s1;
    always @(posedge clk_50mhz) begin sw_s0 <= sw; sw_s1 <= sw_s0; end

    // 1초 타이머
    reg [31:0] cnt;
    reg        start;
    reg  [7:0] cur;
    wire       busy, done;
    always @(posedge clk_50mhz) begin
        if (rst_p) begin cnt <= 0; start <= 1'b0; cur <= 8'h00; end
        else begin
            start <= 1'b0;
            if (cnt == PERIOD - 1) begin
                cnt <= 0;
                if (!busy) begin start <= 1'b1; cur <= sw_s1; end
            end else cnt <= cnt + 1;
        end
    end

    spi_master_tx #(.HALF_DIV(HALF), .NBYTES(1)) u_spi (
        .clk(clk_50mhz), .rst_p(rst_p), .start(start), .tx_data(sw_s1),
        .busy(busy), .done(done), .sck(spi_sck), .mosi(spi_mosi), .cs_n(spi_cs_n));

    // 보낸 값 기록: start 때 바이트, done 때 줄 끝
    spi_uart_log #(.DIV(DIV), .P0("T"), .P1("X")) u_log (
        .clk(clk_50mhz), .rst_p(rst_p),
        .byte_valid(start), .byte_data(sw_s1),
        .frame_end(done), .frame_abort(1'b0),
        .uart_txd(uart_txd), .overflow());

    assign led = cur;
endmodule
```

⑤ 시뮬레이션

TB(`LAB7-02/sim/tb.sv`)는 STM32 슬레이브처럼 SCK 상승 에지에서 MOSI를 읽고, 모든 에지마다 규칙을 검사한다. 송신 간격만 1 s → 10 ms로 줄였다(매개변수 `PERIOD` , 보드 빌드는 1 s).

검사	내용
(a)	CS가 내려가고 올라가는 순간 SCK = LOW
(b)	SCK는 CS LOW일 때만 움직임
(c)	MOSI는 CS 하강 또는 SCK 하강과 같은 시각에만 바뀜
(d)	CS가 내려간 직후 MOSI = 최상위 비트
(e)	한 프레임 상승 에지 8번, 읽은 값 = 그때 스위치 값
(f)	SCK 주기 측정 → 1 MHz 이하
(g)	CS → 첫 상승, 마지막 하강 → CS, 프레임 간격 측정
(h)	FPGA 자신의 UART 줄 "TX=xx" = 보낸 값

XSim 결과

```
spi_tx_top 기본 송신 간격 = 50000000 클록 = 1.000 s (TB에서는 500000 클록)
10018.010 us  MCU(슬레이브) 수신 = 0x55  (CS LOW 17.00 us, CS->첫 상승 1.00 us, 마지막 하강->CS 1.00 us)
LED=01010101
17240.431 us  PC(FPGA COM) <- "TX=55"
20001.010 us  프레임 간격 10.000 ms
20018.010 us  MCU(슬레이브) 수신 = 0xa5  (CS LOW 17.00 us, CS->첫 상승 1.00 us, 마지막 하강->CS 1.00 us)
LED=10100101
27240.431 us  PC(FPGA COM) <- "TX=A5"
30001.010 us  프레임 간격 10.000 ms
30018.010 us  MCU(슬레이브) 수신 = 0x00  (CS LOW 17.00 us, CS->첫 상승 1.00 us, 마지막 하강->CS 1.00 us)
LED=00000000
37240.431 us  PC(FPGA COM) <- "TX=00"
40001.010 us  프레임 간격 10.000 ms
40018.010 us  MCU(슬레이브) 수신 = 0xff  (CS LOW 17.00 us, CS->첫 상승 1.00 us, 마지막 하강->CS 1.00 us)
LED=11111111
47240.431 us  PC(FPGA COM) <- "TX=FF"
50001.010 us  프레임 간격 10.000 ms
50018.010 us  MCU(슬레이브) 수신 = 0x3c  (CS LOW 17.00 us, CS->첫 상승 1.00 us, 마지막 하강->CS 1.00 us)
LED=00111100
57240.431 us  PC(FPGA COM) <- "TX=3C"
60001.010 us  프레임 간격 10.000 ms
```

```
60018.010 us  MCU(슬레이브) 수신 = 0x81  (CS LOW 17.00 us, CS->첫 상승 1.00 us, 마지막 하강->CS 1.00 us)
LED=10000001
67240.431 us  PC(FPGA COM) <- "TX=81"
SCK 주기 2000.0~2000.0 ns -> 500.0 kHz (조건 1 MHz 이하)
CS->첫 상승 최소 1000.0 ns, 마지막 하강->CS 최소 1000.0 ns
PASS spi_tx_top: 6 frames, mode 0 rules (a)-(h) ok, SCK 500 kHz
```

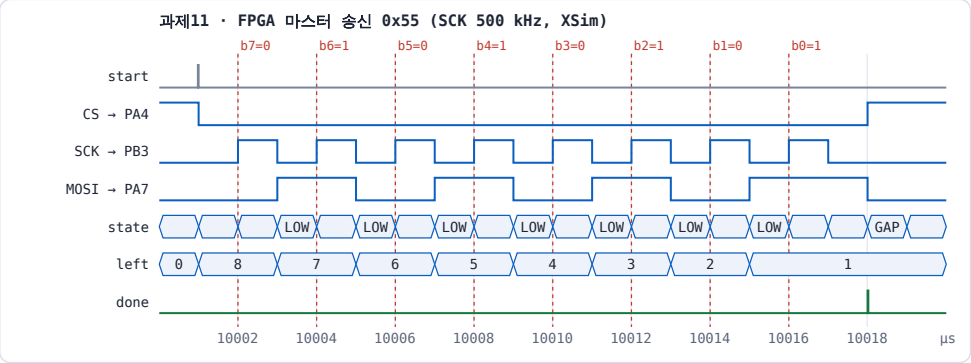


그림 4. FPGA 송신 0x55. CS가 내려가는 순간 MOSI에 b7(=0)이 이미 나와 있고, 이후 MOSI는 SCK 하강 에지에서만 바뀐다. 상승 에지(빨간 점선) 8번에서 읽히는 값이 0 1 0 1 0 1 0 1. left는 지금 나가 있는 비트를 포함한 남은 비트 수.

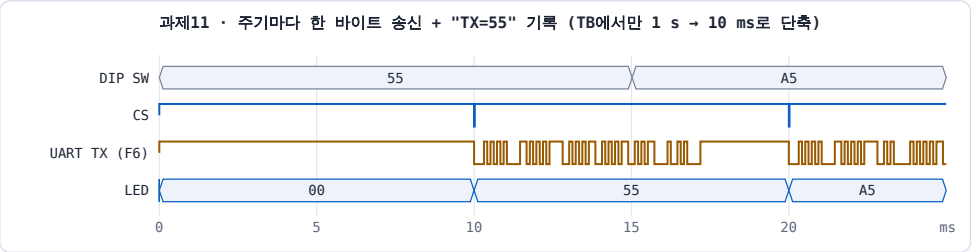


그림 5. 주기마다 한 프레임(CS 펄스)을 보내고, 보낸 값을 F6로 "TX=55"처럼 기록한다. DIP 스위치를 0x55 → 0xA5로 바꾸면 다음 프레임부터 값이 바뀐다.

MCU 쪽 공동 시뮬레이션: TB의 슬레이브 모델이 받은 바이트(`sim/fpga_spi_tx.hex`)를 과제9의 MCU 수신 코드 모의 실행에 넣었다. 수신 MCU의 VCP 출력 예상:

```
# SPI1 init: MASTER, NSS=SOFT, CPOL=0 CPHA=0 (mode 0), 8-bit, MSB first, prescaler=128
# SCK = 100 MHz / 128 = 781.25 kHz
# SPI1 init: SLAVE, NSS=HARD_INPUT, CPOL=0 CPHA=0 (mode 0), 8-bit, MSB first, prescaler=128
# PA4 init: mode=AF_PP pull=UP AF=5
RX=55
RX=A5
RX=00
RX=FF
RX=3C
RX=81
RX timeout: check sender, SCK, MOSI, CS and GND
# PASS: 0 check failures
```

⑥ 합성 · 구현 결과 (`build7 2`)

항목	값	설명
Slice LUTs	207 (0.43%)	1초 카운터 32비트 + 송신기 + 기록기
Slice Registers	202	
Bonded IOB	22	과제10의 14 + DIP 스위치 8
WNS / WHS	**+14.323 ns** / +0.082 ns	
비트스트림	`LAB7_bit/LAB7-02_spi_tx.bit`	

⑦ 보드 실습 계획

1. `prog7 2` → DIP 스위치 0x55(SW2 · SW4 · SW6 · SW8 ON).
2. **MCU 연결 전에** 오실로스코프로 F8 · E8 · A7 확인: 시간축 2 µs/칸, CS 하강 트리거, Single. SCK 주기 2 µs, 상승 에지 8번, CS LOW 17 µs.
3. MCU에 `mcu7 task9_rx -Flash` → Serial Monitor에 `RX timeout` 이 나오는지 확인 → F8 → PB3, E8 → PA7, **A7 → PA4**, GND 연결.
4. MCU 화면 `RX=55` 1초마다 확인, 스위치를 A5 · 00 · FF로 바꿔 반복 → 영상(스위치 · LED · Serial Monitor가 함께 보이게).

5. 실험 5 · STM32_3 과제12 — FPGA → FPGA 송수신 확인

C조와 D조의 FPGA 두 대를 연결해, 한쪽은 과제11 송신 회로, 다른 쪽은 과제10 수신 회로로 값을 주고받고 수신 FPGA의 UART로 확인한 뒤 역할을 바꾼다.

① 목적

직접 만든 송신기와 수신기가 서로 통하는지 확인한다. 두 보드는 각자 다른 수정 발진기를 쓰므로 50 MHz가 정확히 같지 않다.

② 이론

UART는 양쪽이 각자 시계로 비트 폭을 재므로 클록 차이가 쌓이면 틀린다(LAB6에서 0.6% 차이를 확인). SPI는 **마스터가 SCK로 박자를 직접 보내기** 때문에, 슬레이브의 클록이 조금 빠르거나 느려도 SCK 에지를 찾는 샘플링 속도만 충분하면 된다. 수신기는 SCK 반주기(1 µs = 50클록)마다 에지를 찾으므로 ±100 ppm(0.01%)은 전혀 영향이 없다.

③ 연결

신호	송신 보드 (spi_tx_top)	수신 보드 (spi_rx_top)
SCK	F8 출력	F8 입력
MOSI	E8 출력	E8 입력
CS	A7 출력	A7 입력
GND	J10 GND	J10 GND

역할 교대 = **비트스트림만 서로 바꾼다**. 핀이 같아서 배선은 그대로 둔다. 두 보드 모두 PORT A ENABLE OFF.

④ 시뮬레이션

TB(`LAB7-03/sim/tb.sv`)는 과제11 회로(보드 A)와 과제10 회로(보드 B)를 점퍼선 지연 3 ns로 잇고, 두 클록을 +100 ppm / −100 ppm으로 다르게(위상도 7.3 ns 어긋나게) 둔다. `+define+SWAP` 은 클록 차이를 반대로 한 역할 교대판이다. 두 보드의 UART 줄을 모아 A의 "TX=xx"와 B의 "RX=xx" 값이 같은 순서로 일치하는지 검사한다.

정방향 (XSim)

```
보드 A(마스터) 클록 100 ppm, 보드 B(슬레이브) 클록 -100 ppm, 선 지연 3 ns
17238.806 us  보드 A COM <- "TX=55"
17255.125 us  보드 B COM <- "RX=55"   LED=01010101
27237.806 us  보드 A COM <- "TX=A5"
27254.125 us  보드 B COM <- "RX=A5"   LED=10100101
37236.806 us  보드 A COM <- "TX=00"
37253.125 us  보드 B COM <- "RX=00"   LED=00000000
47235.806 us  보드 A COM <- "TX=FF"
47252.125 us  보드 B COM <- "RX=FF"   LED=11111111
57234.806 us  보드 A COM <- "TX=C3"
57251.124 us  보드 B COM <- "RX=C3"   LED=11000011
PASS FPGA->FPGA: 5 frames, B 수신 = A 송신
```

역할 교대 (XSim)

```
보드 A(마스터) 클록 -100 ppm, 보드 B(슬레이브) 클록 100 ppm, 선 지연 3 ns
17242.057 us  보드 A COM <- "TX=55"
17255.872 us  보드 B COM <- "RX=55"   LED=01010101
27243.057 us  보드 A COM <- "TX=A5"
27256.872 us  보드 B COM <- "RX=A5"   LED=10100101
37244.057 us  보드 A COM <- "TX=00"
37257.872 us  보드 B COM <- "RX=00"   LED=00000000
47245.057 us  보드 A COM <- "TX=FF"
47258.871 us  보드 B COM <- "RX=FF"   LED=11111111
57246.057 us  보드 A COM <- "TX=C3"
57259.871 us  보드 B COM <- "RX=C3"   LED=11000011
PASS FPGA->FPGA (역할 교대): 5 frames, B 수신 = A 송신
```

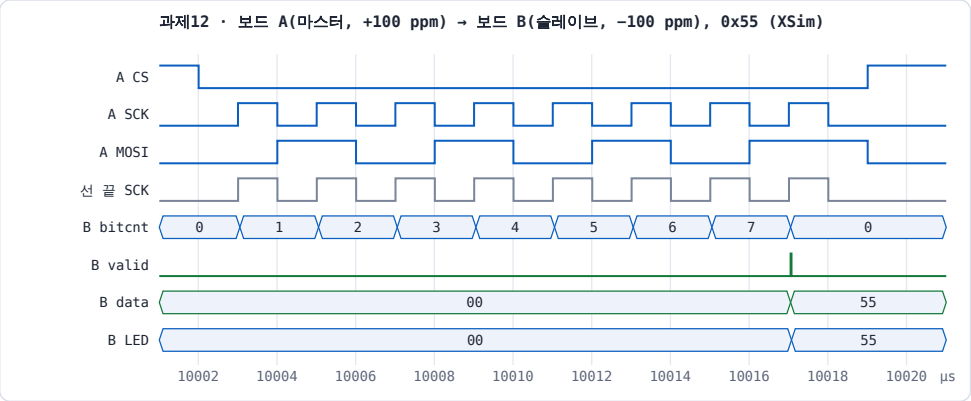


그림 6. 보드 A가 보낸 0x55를 보드 B가 받는 모습. 선 끝 SCK는 3 ns 늦게 도착하고, 보드 B는 자기 클록으로 동기화한 뒤 8번째 에지에서 data = 55를 낸다.

⑤ 보드 실습 계획

1. C조 `prog7 2` (송신), D조는 우리 `LAB7-01_spi_rx.bit`를 올리거나 D조 자신의 수신 회로를 쓴다.
2. GND 먼저, F8 · E8 · A7 연결. 수신 보드 FTDI → COM 포트에서 `RX=55`, 송신 보드 LED와 비교.
3. 비트스트림을 바꿔 역할 교대 → 영상.

고찰

1. **SPI가 UART보다 단순한 이유.** UART는 양쪽이 같은 속도를 약속하고 각자 시계로 비트 가운데를 맞춰야 한다(LAB6 수신기는 1.5비트 기다렸다 읽는 상태 기계). SPI는 클록 선을 따로 보내므로, 수신기는 "SCK가 올라갔나?"만 보면 된다. 그 대가로 선이 하나 더(SCK), 장치마다 CS가 하나씩 더 필요하다.
2. **클록을 하나로 유지한 설계.** SCK를 FPGA 클록으로 쓰면 회로는 더 작아 보이지만, 받은 바이트를 50 MHz 쪽 UART로 넘길 때 두 클록 영역 사이 문제가 다시 생긴다. 모든 입력을 50 MHz로 샘플링하니 회로 전체가 하나의 클록이 되었고, 타이밍 보고서도 하나의 클록만 분석하면 됐다(WNS +14.9 ns). 대신 받을 수 있는 SCK 속도가 약 8 MHz로 제한되며, 이번 실습(≤1 MHz)에는 충분하다.
3. **CS가 프레임의 경계를 정한다.** SCK 선에 잡음이 들어가 비트 하나가 더 세어지면 그 뒤 모든 바이트가 밀린다. CS가 올라갈 때 카운터를 0으로 되돌리기 때문에 잘못은 그 프레임 하나로 끝난다. TB의 "3비트 후 CS 해제" 시나리오가 이것을 확인하고, 덜 찬 프레임은 "RX=?"로 드러나게 해서 당일 배선 불량을 바로 알 수 있게 했다.
4. **result = 0은 전달 확인이 아니다.** MISO를 쓰지 않으므로 마스터는 상대가 받았는지 알 방법이 없다. 그래서 과제10은 FPGA UART, 과제9 · 11은 수신 MCU의 VCP처럼 **받는 쪽의 출력**으로만 판단하도록 모든 과제를 구성했다.
5. **시뮬레이션을 서로 연결했다.** MCU 송신 코드의 모의 실행 출력 → FPGA 수신 TB 입력, FPGA 송신 TB 출력 → MCU 수신 코드 모의 실행 입력으로 이어 붙였다. 각자 따로 맞는 것이 아니라 실제 연결 순서대로 데이터가 흘러가는지 확인한 것이다.
6. **두 시뮬레이터에서 생긴 차이.** 회로 결과는 Icarus와 XSim이 같았지만, TB 출력에서 두 가지가 달랐다. ① XSim은 `%+f` 출력 형식을 지원하지 않아 로그 한 줄이 깨졌다 → 정수 출력으로 바꿈. ② 선지연에 `real` 변수를 쓰는 것은 표준상 상수여야 해서 `localparam`으로 바꿨다. 또 PowerShell이 시뮬레이터의 UTF-8 한글을 CP949로 읽어 결과 파일이 깨졌고, 도구에서 콘솔 인코딩을 UTF-8로 지정해 해결했다.
7. **모델의 한계.** MCU의 CS → 첫 SCK 간격(2 µs)과 끝 간격(1.5 µs)은 HAL 코드 흐름으로 짐작한 값이다. 수신 회로는 이 간격이 0이 아니기만 하면 동작하도록 만들었고, 실제 값은 과제8 오실로스코프로 재서 실험 후 보고서에서 비교한다.

참고문헌

1. [1] D. M. Harris and S. L. Harris, *Digital Design and Computer Architecture*, 2nd ed. Waltham, MA, USA: Morgan Kaufmann, 2013. ISBN 978-0-12-394424-5. — Ch. 3.5.5 동기화기(2단 플립플롭과 준안정), Ch. 8.6.3 직렬 I/O의 SPI(SCK·MOSI·MISO, 클록 극성·위상)와 UART 비교.
2. [2] STMicroelectronics, *stm32g4xx-hal-driver*, GitHub, https://github.com/STMicroelectronics/stm32g4xx-hal-driver, 태그 v1.2.7. — `stm32g4xx_hal_spi.c`의 `HAL_SPI_Init` · `HAL_SPI_DeInit` · `HAL_SPI_Transmit` · `HAL_SPI_Receive` (타임아웃 시 `HAL_TIMEOUT`), `stm32g4xx_hal_spi.h`의 `SPI_NSS_HARD_INPUT` · `SPI_PHASE_1EDGE`.

수업 자료(인용 출처 수에 넣지 않음): Glaysia/ece2_474 v1.1.0의 docs/STM32_3.md(과제8~12, 제공 MCU 코드) · Core/Src/main.c(MX_SPI1_Init: 프리스케일러 128, 모드 0), 실습 교재 「Combo II DLD 장비를 이용한 디지털 논리 회로 설계」 p.73(확장 포트 BDIO 핀표). LAB3-07 · LAB6 UART 회로(본인 저장소).